

PROMETHEUS

+

GRAFANA

Taller Práctico · Bootcamperu — Escuela de Tecnología

Taller de Observabilidad

Guía del Proyecto

Proyecto integrador: **taller-observabilidad**

4 horas

AWS

Intermedio

CONTENIDO

- | | | | |
|----|--------------------------|----|---------------------------------|
| 01 | Bienvenida | 06 | El proyecto que vas a construir |
| 02 | Objetivos de aprendizaje | 07 | Checkpoints funcionales |
| 03 | Requisitos previos | 08 | Dashboards y alertas |
| 04 | Paso 1 — Preparación | 09 | Reglas del juego |
| 05 | Arquitectura objetivo | 10 | Referencias oficiales |

01 Bienvenida

Has completado el taller de Infraestructura como Código y ya sabes desplegar una aplicación web de dos capas en AWS con Terraform y Ansible. Ahora vamos un paso más allá: hacer que esa aplicación sea **observable**.

En 3.5 horas efectivas vas a construir, desde cero, un stack de monitoreo profesional: Prometheus para métricas, Grafana para dashboards, Alertmanager para notificaciones, Loki para logs centralizados, y YACE para traer métricas de AWS CloudWatch al mismo panel. Todo instalado manualmente desde paquetes oficiales y luego automatizado con Ansible.

Al terminar vas a tener dashboards con las cuatro Golden Signals de tu app, alertas que notifican a un webhook cuando hay anomalías, y logs correlacionados con métricas. Todo como código, reproducible, y desmontable con un comando.

Alcance

Este taller **no cubre** la teoría base de observabilidad (Golden Signals, RED, USE, tipos de métricas, tracing). Esos conceptos se asumen conocidos del temario previo del bootcamp.

02 Objetivos de aprendizaje

Al terminar este taller vas a poder, sin ayuda externa, hacer las cosas concretas listadas abajo. Son objetivos verificables: si no puedes hacerlos, el taller no cumplió su propósito.

- Instalar Prometheus y Grafana manualmente desde paquete oficial en una instancia Ubuntu 22.04, configurarlos como servicios de systemd y validar que están corriendo.
- Instrumentar una aplicación Flask existente para exponer métricas RED (rate, errors, duration) en un endpoint /metrics.
- Escribir y validar reglas de scraping en `prometheus.yml`, targets estáticos y uso del SD de EC2 para autodescubrimiento.
- Construir un dashboard de Golden Signals en Grafana escribiendo las consultas PromQL panel por panel.
- Configurar Loki + Promtail para centralizar logs de nginx y Flask, y correlacionarlos con métricas en el mismo panel de Grafana.
- Definir reglas de alerta en Alertmanager que notifican a un webhook cuando una métrica pasa un umbral.
- Integrar métricas de AWS CloudWatch mediante YACE con IAM Role y Instance Profile, sin hardcodear credenciales.
- Automatizar todo lo anterior como un rol de Ansible reutilizable, ejecutable con un solo playbook sobre infraestructura limpia.

03 Requisitos previos

3.1 Conocimientos

- Haber completado el **Taller de Infraestructura como Código** (Terraform + Ansible) de Bootcamperu.
- Manejo básico de Linux CLI, SSH y systemd.
- Conceptos base de observabilidad: Golden Signals, RED, USE, counter, gauge, histogram, PromQL mínimo.

3.2 Herramientas en tu laptop

Herramienta	Versión mínima	Verificación rápida
Terraform	≥ 1.0	<code>terraform -version</code>
Ansible	≥ 2.12 (se recomienda 2.20+)	<code>ansible --version</code>
AWS CLI v2	cualquiera reciente	<code>aws --version</code>
Git	cualquiera	<code>git --version</code>
curl, jq	cualquiera	<code>jq --version</code>
Editor con YAML/HCL	—	VS Code recomendado

3.3 Cuenta y credenciales AWS

- Cuenta AWS con permisos para crear VPC, EC2, IAM Role e Instance Profile, Security Groups y leer CloudWatch.
- Access Key y Secret Key configurados con `aws configure`, o bien un perfil con SSO que resuelva a esa cuenta.
- Región por defecto: **us-east-1**.
- Key pair SSH creado y su `.pem` descargado localmente.

04 Paso 1 — Preparación (antes del taller)

Importante

Este paso debe estar **100 % completo** antes de que empiece el taller. Los 210 minutos efectivos de clase están reservados para observabilidad, no para troubleshooting del stack base. Resérvate 30–45 minutos el día anterior para dejarlo listo.

4.1 Objetivo

Llegar al inicio del taller con el proyecto base **taller-iac** clonado, inicializado y con la infraestructura corriendo en tu cuenta AWS.

4.2 Acciones

4.2.1 — Instalar las herramientas de §3.2

Sigue la documentación oficial de cada herramienta para tu sistema operativo. El README.md del repositorio taller-iac incluye enlaces.

4.2.2 — Configurar credenciales AWS

```
aws configure
# pedirá Access Key, Secret Key, región y formato
```

Valida con `aws sts get-caller-identity` — debe devolver tu cuenta y tu ARN.

4.2.3 — Crear el bucket S3 del state remoto

```
aws s3 mb s3://bootcamperu-tf-state --region us-east-1
```

El archivo proveedores.tf de taller-iac está configurado para usar este bucket como backend remoto de Terraform. Si el bucket no existe, `terraform init` falla.

4.2.4 — Clonar el repositorio base

```
git clone https://github.com/ScrambledBits/taller-iac.git
# entra al directorio terraform/ antes del init
```

4.2.5 — Inicializar y aplicar

```
terraform init
# descarga providers y conecta con S3
```

```
terraform plan
# revisa qué se va a crear
```

```
terraform apply
# crea VPC + 2 EC2 + ejecuta Ansible · ~3-5 min
```

El apply primero crea los recursos en AWS, luego un provisioner genera el inventario de Ansible y ejecuta el playbook que instala nginx en el frontend y Flask + systemd en el backend.

4.2.6 — Validar que la aplicación responde

```
FE=$(terraform output -raw frontend_public_ip)
```

```
curl http://$FE/api/health
```

```
curl http://$FE/api/hello
```

Ambos curl deben devolver JSON con status: ok. También puedes abrir `http://$FE` en el navegador.

4.3 Criterio de "preparado"

- `aws sts get-caller-identity` devuelve tu identidad correctamente.
- El bucket `s3://bootcamperu-tf-state` existe en tu cuenta.
- `terraform apply` terminó sin errores y los outputs muestran `frontend_public_ip` y `backend_private_ip`.
- Los dos endpoints `/api/health` y `/api/hello` devuelven JSON válido.

05 Arquitectura objetivo

El proyecto base **taller-iac** despliega una aplicación web de dos capas en una VPC propia: un frontend con nginx en subnet pública y un backend con Flask en subnet privada. En este taller añades una tercera instancia, **monitoring**, que hospeda todo el stack de observabilidad.

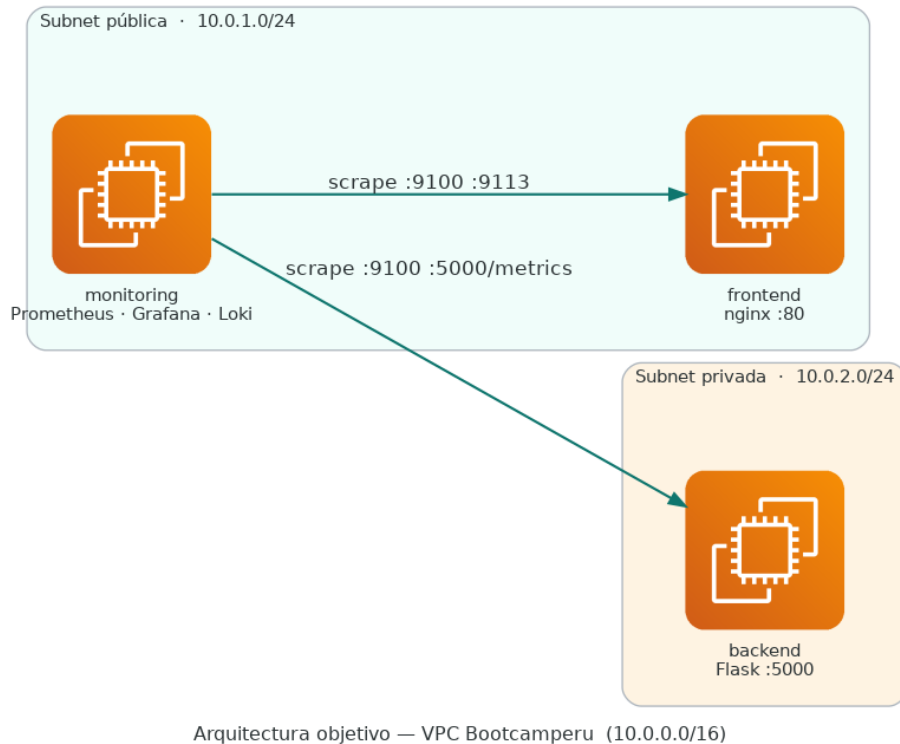


Figura 1 — Arquitectura objetivo. El host monitoring scrapea por IPs privadas tanto al frontend como al backend dentro de la VPC.

5.1 Flujo de datos

Prometheus pulsa (scrapea) periódicamente a los exporters expuestos en cada host. Grafana consulta Prometheus (métricas) y Loki (logs) como datasources separados, pero los muestra en el mismo panel para correlacionar. Alertmanager recibe alertas desde Prometheus según las reglas definidas y las enruta al webhook configurado.

5.2 Componentes y sus puertos

Componente	Host	Puerto	Rol
Prometheus	monitoring	9090	Scraping + almacén de series
Grafana	monitoring	3000	UI de dashboards
Alertmanager	monitoring	9093	Routing de alertas → webhook
Loki	monitoring	3100	Almacén de logs centralizados
node_exporter	frontend + backend	9100	Métricas de host
nginx exporter	frontend	9113	Métricas de nginx (stub_status)
Flask /metrics	backend	5000	Métricas de la app
Promtail	frontend + backend	—	Shipper de logs hacia Loki
YACE	monitoring	5000 (interno)	Exporter de CloudWatch

06 El proyecto que vas a construir

6.1 Dos proyectos, dos directorios

Al final del taller tendrás dos proyectos convivientes en tu máquina. Son proyectos separados, con sus propios directorios de terraform y ansible, y con estados de Terraform independientes.

Proyecto	Origen	Rol
taller-iac	Clonado del repo de Bootcamperu	Despliega la app (frontend + backend).
taller-observabilidad	Creado desde cero en clase	Despliega el stack de monitoreo e instrumenta la app.

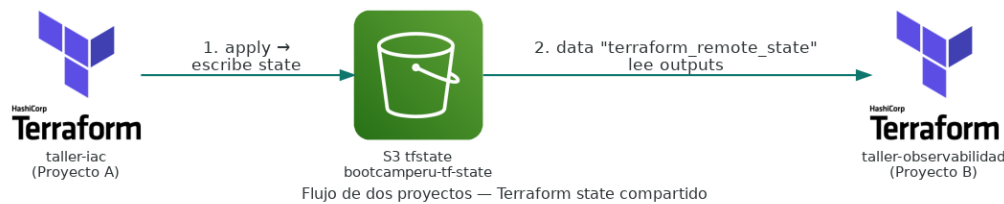


Figura 2 — Flujo de dos proyectos. *taller-observabilidad* consume el state remoto de *taller-iac* vía data "terraform_remote_state".

6.2 Estructura del nuevo proyecto

taller-observabilidad replica la convención del proyecto base: una carpeta `terraform/` y una `ansible/` independientes.

Terraform

- `proveedores.tf` — provider AWS y backend S3 con key distinta del proyecto base.
- `datos.tf` — data source `terraform_remote_state` que lee el state de *taller-iac*.
- `computo.tf` — EC2 monitoring con Instance Profile para YACE.
- `seguridad.tf` — reglas de ingress desde el SG monitoring hacia el SG común de *taller-iac*.
- `aprovisionamiento.tf` — genera el inventario de Ansible con los 3 hosts.

Ansible

- `roles/monitoring` — instala Prometheus, Grafana, Alertmanager, Loki, YACE.
- `roles/nodos` — instala `node_exporter` y `Promtail` en frontend y backend.
- `roles/frontend_obs` — instala `nginx-prometheus-exporter`.
- `roles/backend_obs` — instrumenta la app Flask con `prometheus-flask-exporter`.

6.3 Cambio mínimo al proyecto base

El repositorio *taller-iac* solo se toca en un archivo: `salidas.tf`, donde se agregan dos outputs nuevos para que el proyecto de observabilidad pueda leerlos:

- `comun_security_group_id` — para abrir puertos de scraping vía referencia SG-a-SG desde el segundo proyecto.
- `frontend_private_ip` — para que Prometheus scrapee el frontend por IP privada.

Operación

Después de añadir los outputs, ejecuta `terraform apply` una vez más en **taller-iac**. Solo actualiza el state; no recrea recursos.

07 Checkpoints funcionales

El taller está dividido en bloques, y cada bloque tiene un checkpoint que debe verse verde antes de avanzar. Estos son los ocho hitos del proyecto:

#	Checkpoint	Cómo lo valido
CP-1	Prometheus corriendo y scrapeándose a sí mismo	<code>curl http://MON:9090/-/healthy + Status → Targets</code>
CP-2	node_exporter UP en frontend y backend	Los tres targets aparecen UP en la UI de Prometheus
CP-3	Flask expone métricas en /metrics	<code>curl BE:5000/metrics grep flask_http</code>
CP-4	Grafana abre y muestra un dashboard Golden Signals	Login admin funciona, p95 de /api/health se grafica
CP-5	Alerta dispara a webhook (test con webhook.site)	El webhook recibe el payload con annotations
CP-6	Loki recibe logs de nginx y Flask	Query en Explore muestra líneas con labels job=nginx/flask
CP-7	Métricas CloudWatch visibles vía YACE	Panel muestra <code>aws_ec2_cpuutilization_average</code>
CP-8	Todo el stack se levanta con un solo ansible-playbook	<code>terraform destroy + apply + playbook desde cero</code> pasa los 7 CPs

08 Dashboards y alertas

8.1 Dashboard Golden Signals — WebStack

El dashboard principal que construyes en clase. Lo armas panel por panel para que entiendas cada expresión PromQL que escribes.

Fila	Métrica	Expresión PromQL
1	Latencia p95 por endpoint	<code>histogram_quantile(0.95, sum(rate(flask_http_request_duration_seconds_bucket[5m])) by (le, path))</code>
2	Tráfico status por	<code>sum(rate(flask_http_request_total[1m])) by (status)</code>
3	Tasa errores de	<code>sum(rate(flask_http_request_total{status=~"5.."}[5m])) / sum(rate(flask_http_request_total[5m]))</code>
4	Saturación CPU	<code>1 - avg by (instance)(rate(node_cpu_seconds_total{mode="idle"}[5m]))</code>

8.2 Reglas de alerta mínimas

- **HighErrorRate** — tasa de 5xx > 5 % por 2 minutos.
- **LatencyP95High** — p95 de /api/* > 500 ms por 5 minutos.
- **InstanceDown** — up == 0 por 1 minuto.
- **DiskSpaceLow** — node_filesystem_avail_bytes < 20 % por 10 minutos.

Nota

Todas las alertas se enrutan al mismo webhook de prueba. En producción definirías receivers separados (Slack, PagerDuty, email) según severidad.

09 Reglas del juego

9.1 Stack y método de instalación

- Todo el stack se instala **manualmente** desde paquete oficial (APT para Grafana) o binario tar.gz (Prometheus, Alertmanager, Loki, node_exporter, nginx-prometheus-exporter, YACE, Promtail).
- No se usa Docker en ningún paso del taller. Cada servicio corre como systemd unit con su propio usuario de sistema sin shell.
- Después del instalado a mano, lo automatizas con Ansible. El rol final es la "versión memorizada" del procedimiento manual.

9.2 Gestión de versiones

Las versiones concretas de cada componente viven como variables en `ansible/group_vars/all.yml`. Esto permite actualizar el stack cambiando un solo archivo sin tocar tareas.

9.3 Criterio de código

- Nada de `latest` en versiones.
- Nada de credenciales hardcoded: YACE usa IAM Role + Instance Profile.
- Nada de `ansible.posix.firewalld` o `iptables` a mano: los puertos se gestionan en Security Groups de AWS.
- Todo cambio al proyecto base `taller-iac` debe ser aditivo (nuevos outputs, nuevos tags). Nada se modifica destructivamente.

10 Referencias oficiales

Usa siempre la documentación oficial como fuente primaria. Estos enlaces son tu referencia durante y después del taller:

Componente	Documentación / Repo
Repo del proyecto base	github.com/ScrambledBits/taller-iac
Prometheus	prometheus.io/docs · github.com/prometheus/prometheus
Grafana	grafana.com/docs · apt.grafana.com
Alertmanager	prometheus.io/docs/alerting/latest/configuration
Loki	grafana.com/docs/loki
Node Exporter	github.com/prometheus/node_exporter
nginx-prometheus-exporter	github.com/nginx/nginx-prometheus-exporter
prometheus-flask-exporter	github.com/rycus86/prometheus_flask_exporter
YACE	github.com/prometheus-community/yet-another-cloudwatch-exporter
Terraform <code>remote_state</code>	developer.hashicorp.com/terraform/language/state/remote-state-data